Parameterized Queries

This section covers a topic that is skipped in most SQL textbooks: *parameterized queries* and *bind parameters*.

Bind parameters—also called dynamic parameters or bind variables—are an alternative way to pass data to the database. Instead of putting the values directly into the SQL statement, you just use a placeholder like `?`, `:name` or `@name` and provide the actual values using a separate API call.

There is nothing bad about writing values directly into ad-hoc statements; there are, however, two good reasons to use bind parameters in programs:

Security
: Bind variables are the best way to prevent SQL injection[1].

Performance
: Databases with an execution plan cache like SQL Server and the Oracle database can reuse an execution plan when executing the same statement multiple times. It saves effort in rebuilding the execution plan but works only if the SQL statement is *exactly* the same. If you put different values into the SQL statement, the database handles it like a different statement and recreates the execution plan.

  When using bind parameters you do not write the actual values but instead insert placeholders into the SQL statement. That way the statements do not change when executing them with different values.

[1] http://en.wikipedia.org/wiki/SQL_injection

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Naturally there are exceptions, for example if the affected data volume depends on the actual values:

```
99 rows selected.

SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 20;

-------------------------------------------------------------------
|Id | Operation                    | Name        | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |             |   99 |   70 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |   99 |   70 |
|*2 |   INDEX RANGE SCAN           | EMPLOYEE_PK |   99 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("SUBSIDIARY_ID"=20)
```

An index lookup delivers the best performance for small subsidiaries, but a `TABLE ACCESS FULL` can outperform the index for large subsidiaries:

```
1000 rows selected.

SELECT first_name, last_name
  FROM employees
 WHERE subsidiary_id = 30;

-----------------------------------------------------
| Id | Operation         | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT  |           | 1000 |  478 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES | 1000 |  478 |
-----------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("SUBSIDIARY_ID"=30)
```

In this case, the histogram on `SUBSIDIARY_ID` fulfills its purpose. The optimizer uses it to determine the frequency of the subsidiary ID mentioned in the SQL query. Consequently it gets two different row count estimates for both queries.

The subsequent cost calculation will therefore result in two different cost values. When the optimizer finally selects an execution plan it takes the plan with the lowest cost value. For the smaller subsidiary, it is the one using the index.

The cost of the `TABLE ACCESS BY INDEX ROWID` operation is highly sensitive to the row count estimate. Selecting ten times as many rows will elevate the cost value by that factor. The overall cost using the index is then even higher than a full table scan. The optimizer will therefore select the other execution plan for the bigger subsidiary.

When using bind parameters, the optimizer has no concrete values available to determine their frequency. It then just assumes an equal distribution and always gets the same row count estimates and cost values. In the end, it will always select the same execution plan.



Tip

Column histograms are most useful if the values are not uniformly distributed.

For columns with uniform distribution, it is often sufficient to divide the number of distinct values by the number of rows in the table. This method also works when using bind parameters.

If we compare the optimizer to a compiler, bind variables are like program variables, but if you write the values directly into the statement they are more like constants. The database can use the values from the SQL statement during optimization just like a compiler can evaluate constant expressions during compilation. Bind parameters are, put simply, not visible to the optimizer just as the runtime values of variables are not known to the compiler.

From this perspective, it is a little bit paradoxical that bind parameters can improve performance if not using bind parameters enables the optimizer to always opt for the best execution plan. But the question is at what price? Generating and evaluating all execution plan variants is a huge effort that does not pay off if you get the same result in the end anyway.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



Tip

Not using bind parameters is like recompiling a program every time.

Deciding to build a specialized or generic execution plan presents a dilemma for the database. Either effort is taken to evaluate all possible plan variants for each execution in order to always get the best execution plan or the optimization overhead is saved and a cached execution plan is used whenever possible—accepting the risk of using a suboptimal execution plan. The quandary is that the database does not know if the full optimization cycle delivers a different execution plan without actually doing the full optimization. Database vendors try to solve this dilemma with heuristic methods—but with very limited success.

As the developer, you can use bind parameters deliberately to help resolve this dilemma. That is, you should always use bind parameters except for values that *shall* influence the execution plan.

Unevenly distributed status codes like "todo" and "done" are a good example. The number of "done" entries often exceeds the "todo" records by an order of magnitude. Using an index only makes sense when searching for "todo" entries in that case. Partitioning is another example—that is, if you split tables and indexes across several storage areas. The actual values can then influence which partitions have to be scanned. The performance of `LIKE` queries can suffer from bind parameters as well as we will see in the next section.



Tip

In all reality, there are only a few cases in which the actual values affect the execution plan. You should therefore use bind parameters if in doubt—just to prevent SQL injections.

The following code snippets show how to use bind parameters in various programming languages.

C#

Without bind parameters:

```
int subsidiary_id;
SqlCommand cmd = new SqlCommand(
                   "select first_name, last_name"
                 + "  from employees"
                 + " where subsidiary_id = " + subsidiary_id
                 , connection);
```

Using a bind parameter:

```
int subsidiary_id;
SqlCommand cmd =
       new SqlCommand(
                    "select first_name, last_name"
                  + "  from employees"
                  + " where subsidiary_id = @subsidiary_id
                  , connection);
cmd.Parameters.AddWithValue("@subsidiary_id", subsidiary_id);
```

See also: `SqlParameterCollection` class documentation.

Java

Without bind parameters:

```
int subsidiary_id;
Statement command = connection.createStatement(
                    "select first_name, last_name"
                  + "  from employees"
                  + " where subsidiary_id = " + subsidiary_id
                  );
```

Using a bind parameter:

```
int subsidiary_id;
PreparedStatement command = connection.prepareStatement(
                    "select first_name, last_name"
                  + "  from employees"
                  + " where subsidiary_id = ?"
                  );
command.setInt(1, subsidiary_id);
```

See also: `PreparedStatement` class documentation.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Perl

Without bind parameters:

```
my $subsidiary_id;
my $sth = $dbh->prepare(
                  "select first_name, last_name"
                . "  from employees"
                . " where subsidiary_id = $subsidiary_id"
                );
$sth->execute();
```

Using a bind parameter:

```
my $subsidiary_id;
my $sth = $dbh->prepare(
                  "select first_name, last_name"
                . "  from employees"
                . " where subsidiary_id = ?"
                );
$sth->execute($subsidiary_id);
```

See: Programming the Perl DBI.

PHP

Using MySQL, without bind parameters:

```
$mysqli->query("select first_name, last_name"
             . "  from employees"
             . " where subsidiary_id = " . $subsidiary_id);
```

Using a bind parameter:

```
if ($stmt = $mysqli->prepare("select first_name, last_name"
                           . "  from employees"
                           . " where subsidiary_id = ?"))
{
   $stmt->bind_param("i", $subsidiary_id);
   $stmt->execute();
} else {
  /* handle SQL error */
}
```

See also: `mysqli_stmt::bind_param` class documentation and "Prepared statements and stored procedures" in the PDO documentation.

Ruby

Without bind parameters:

```
dbh.execute("select first_name, last_name"
          + "  from employees"
          + " where subsidiary_id = #{subsidiary_id}");
```

Using a bind parameter:

```
dbh.prepare("select first_name, last_name"
          + "  from employees"
          + " where subsidiary_id = ?");
dbh.execute(subsidiary_id);
```

See also: "Quoting, Placeholders, and Parameter Binding" in the Ruby DBI Tutorial.

The question mark (?) is the only placeholder character that the SQL standard defines. Question marks are positional parameters. That means the question marks are numbered from left to right. To bind a value to a particular question mark, you have to specify its number. That can, however, be very impractical because the numbering changes when adding or removing placeholders. Many databases offer a proprietary extension for named parameters to solve this problem—e.g., using an "at" symbol (`@name`) or a colon (`:name`).



Note

Bind parameters cannot change the structure of an SQL statement.

That means you cannot use bind parameters for table or column names. The following bind parameters do not work:

```
String sql = prepare("SELECT * FROM ? WHERE ?");

sql.execute('employees', 'employee_id = 1');
```

If you need to change the structure of an SQL statement during runtime, use dynamic SQL.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Cursor Sharing and Auto Parameterization

The more complex the optimizer and the SQL query become, the more important execution plan caching becomes. The SQL Server and Oracle databases have features to automatically replace the literal values in a SQL string with bind parameters. These features are called `CURSOR_SHARING` (Oracle) or *forced parameterization* (SQL Server).

Both features are workarounds for applications that do not use bind parameters at all. Enabling these features prevents developers from intentionally using literal values.

Searching for Ranges

Inequality operators such as <, > and **`between`** can use indexes just like the equals operator explained above. Even a `LIKE` filter can—under certain circumstances—use an index just like range conditions do.

Using these operations limits the choice of the column order in multi-column indexes. This limitation can even rule out all optimal indexing options—there are queries where you simply cannot define a "correct" column order at all.

Greater, Less and `BETWEEN`

The biggest performance risk of an `INDEX RANGE SCAN` is the leaf node traversal. It is therefore the golden rule of indexing to keep the scanned index range as small as possible. You can check that by asking yourself where an index scan starts and where it ends.

The question is easy to answer if the SQL statement mentions the start and stop conditions explicitly:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE date_of_birth >= TO_DATE(?, 'YYYY-MM-DD')
   AND date_of_birth <= TO_DATE(?, 'YYYY-MM-DD')
```

An index on DATE_OF_BIRTH is only scanned in the specified range. The scan starts at the first date and ends at the second. We cannot narrow the scanned index range any further.

The start and stop conditions are less obvious if a second column becomes involved:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE date_of_birth >= TO_DATE(?, 'YYYY-MM-DD')
   AND date_of_birth <= TO_DATE(?, 'YYYY-MM-DD')
   AND subsidiary_id  = ?
```

Of course an ideal index has to cover both columns, but the question is in which order?

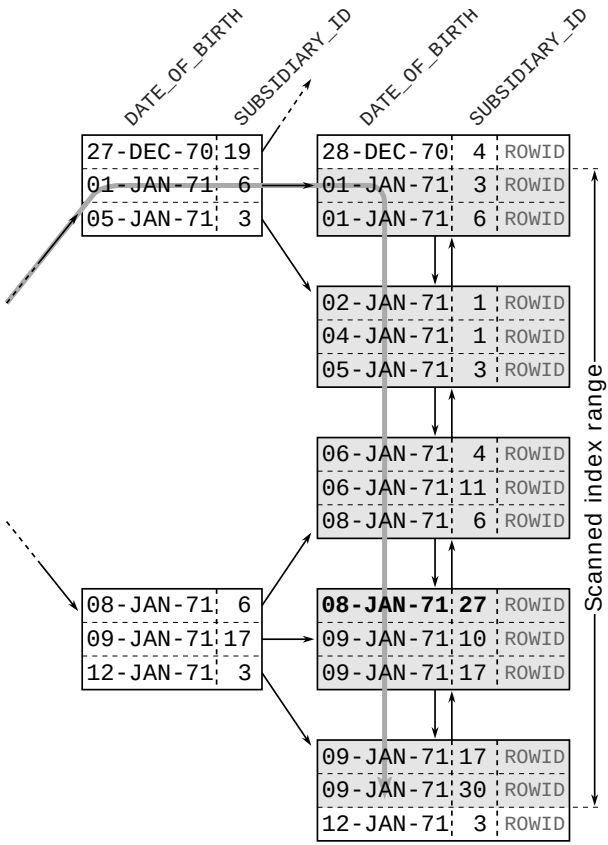
The following figures show the effect of the column order on the scanned index range. For this illustration we search all employees of subsidiary 27 who were born between January 1st and January 9th 1971.

Figure 2.2 visualizes a detail of the index on DATE_OF_BIRTH and SUBSIDIARY_ID—in that order. Where will the database start to follow the leaf node chain, or to put it another way: where will the tree traversal end?

The index is ordered by birth dates first. Only if two employees were born on the same day is the SUBSIDIARY_ID used to sort these records. The query, however, covers a date *range*. The ordering of SUBSIDIARY_ID is therefore useless during tree traversal. That becomes obvious if you realize that there is no entry for subsidiary 27 in the branch nodes—although there is one in the leaf nodes. The filter on DATE_OF_BIRTH is therefore the only condition that limits the scanned index range. It starts at the first entry matching the date range and ends at the last one—all five leaf nodes shown in Figure 2.2.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

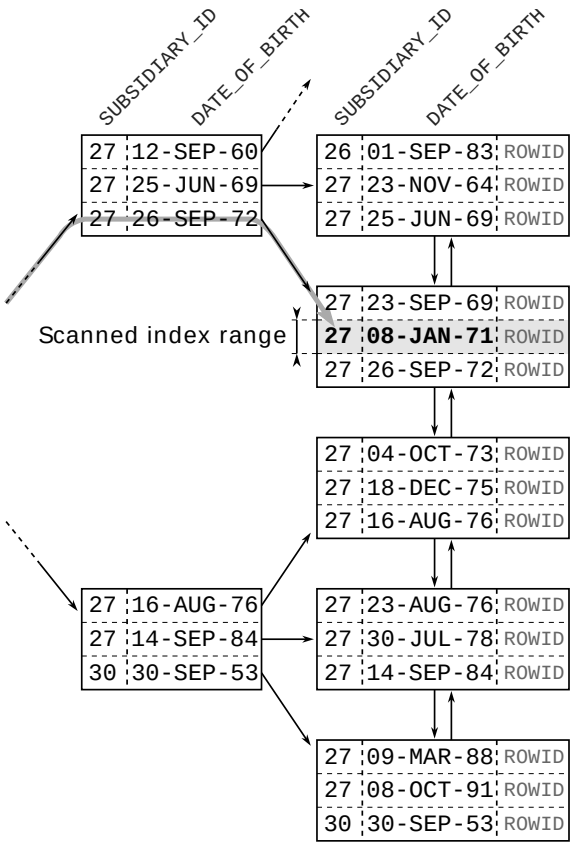
Figure 2.2. Range Scan in DATE_OF_BIRTH, SUBSIDIARY_ID **Index**



The picture looks entirely different when reversing the column order. Figure 2.3 illustrates the scan if the index starts with the SUBSIDIARY_ID column.

The difference is that the equals operator limits the first index column to a single value. Within the range for this value (SUBSIDIARY_ID 27) the index is sorted according to the second column—the date of birth—so there is no need to visit the first leaf node because the branch node already indicates that there is no employee for subsidiary 27 born after June 25th 1969 in the first leaf node.

Figure 2.3. Range Scan in SUBSIDIARY_ID, DATE_OF_BIRTH **Index**



The tree traversal directly leads to the second leaf node. In this case, all **where** clause conditions limit the scanned index range so that the scan terminates at the very same leaf node.



Tip

Rule of thumb: index for equality first—then for ranges.

The actual performance difference depends on the data and search criteria. The difference can be negligible if the filter on DATE_OF_BIRTH is very selective on its own. The bigger the date range becomes, the bigger the performance difference will be.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

With this example, we can also falsify the myth that the most selective column should be at the leftmost index position. If we look at the figures and consider the selectivity of the first column only, we see that both conditions match 13 records. This is the case regardless whether we filter by DATE_OF_BIRTH only or by SUBSIDIARY_ID only. The selectivity is of no use here, but one column order is still better than the other.

To optimize performance, it is very important to know the scanned index range. With most databases you can even see this in the execution plan—you just have to know what to look for. The following execution plan from the Oracle database unambiguously indicates that the EMP_TEST index starts with the DATE_OF_BIRTH column.

```
-------------------------------------------------------------
|Id | Operation                     | Name      | Rows | Cost |
-------------------------------------------------------------
| 0 | SELECT STATEMENT              |           |    1 |    4 |
|*1 |  FILTER                       |           |      |      |
| 2 |   TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    4 |
|*3 |    INDEX RANGE SCAN           | EMP_TEST  |    2 |    2 |
-------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter(:END_DT >= :START_DT)
3 - access(DATE_OF_BIRTH >= :START_DT
       AND DATE_OF_BIRTH <= :END_DT)
    filter(SUBSIDIARY_ID  = :SUBS_ID)
```

The *predicate information* for the INDEX RANGE SCAN gives the crucial hint. It identifies the conditions of the **where** clause either as *access* or as *filter* predicates. This is how the database tells us how it uses each condition.



NOTE

The execution plan was simplified for clarity. The appendix on page 170 explains the details of the "Predicate Information" section in an Oracle execution plan.

The conditions on the DATE_OF_BIRTH column are the only ones listed as access predicates; they limit the scanned index range. The DATE_OF_BIRTH is therefore the first column in the EMP_TEST index. The SUBSIDIARY_ID column is used only as a filter.



Important

The *access predicates* are the start and stop conditions for an index lookup. They define the scanned index range.

Index filter predicates are applied during the leaf node traversal only. They do not narrow the scanned index range.

Appendix A explains how to recognize access predicates in other databases.

The database can use all conditions as access predicates if we turn the index definition around:

```
-------------------------------------------------------------------
| Id | Operation                    | Name      | Rows | Cost |
-------------------------------------------------------------------
|  0 | SELECT STATEMENT             |           |    1 |    3 |
|* 1 |  FILTER                      |           |      |      |
|  2 |   TABLE ACCESS BY INDEX ROWID| EMPLOYEES |    1 |    3 |
|* 3 |    INDEX RANGE SCAN          | EMP_TEST2 |    1 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
1 - filter(:END_DT >= :START_DT)
3 - access(SUBSIDIARY_ID  = :SUBS_ID
       AND DATE_OF_BIRTH >= :START_DT
       AND DATE_OF_BIRTH <= :END_T)
```

Finally, there is the **between** operator. It allows you to specify the upper and lower bounds in a single condition:

```
DATE_OF_BIRTH BETWEEN '01-JAN-71'
                  AND '10-JAN-71'
```

Note that **between** always includes the specified values, just like using the less than or equal to (<=) and greater than or equal to (>=) operators:

```
    DATE_OF_BIRTH >= '01-JAN-71'
AND DATE_OF_BIRTH <= '10-JAN-71'
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

INDEXING LIKE FILTERS

The SQL LIKE operator very often causes unexpected performance behavior because some search terms prevent efficient index usage. That means that there are search terms that can be indexed very well, but others can not. It is the position of the wildcard characters that makes all the difference.

The following example uses the % wildcard in the middle of the search term:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE UPPER(last_name) LIKE 'WIN%D'

-------------------------------------------------------------------
|Id | Operation                    | Name         | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |              |    1 |    4 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    4 |
|*2 |   INDEX RANGE SCAN           | EMP_UP_NAME |    1 |    2 |
-------------------------------------------------------------------
```

LIKE filters can only use the characters *before the first wildcard* during tree traversal. The remaining characters are just filter predicates that do not narrow the scanned index range. A single LIKE expression can therefore contain two predicate types: (1) the part before the first wildcard as an access predicate; (2) the other characters as a filter predicate.

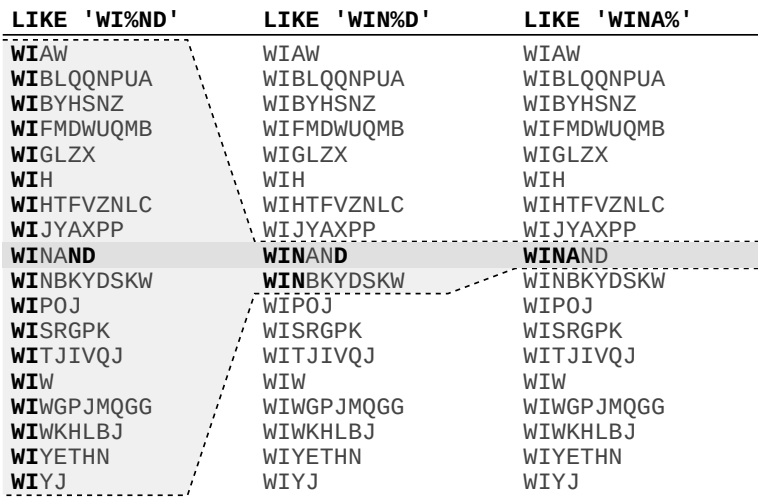


CAUTION

For the PostgreSQL database, you might need to specify an operator class (e.g., varchar_pattern_ops) to use LIKE expressions as access predicates. Refer to "Operator Classes and Operator Families" in the PostgreSQL documentation for further details.

The more selective the prefix before the first wildcard is, the smaller the scanned index range becomes. That, in turn, makes the index lookup faster. Figure 2.4 illustrates this relationship using three different LIKE expressions. All three select the same row, but the scanned index range—and thus the performance—is very different.

Figure 2.4. Various `LIKE` Searches



The first expression has two characters before the wildcard. They limit the scanned index range to 18 rows. Only one of them matches the entire `LIKE` expression—the other 17 are fetched but discarded. The second expression has a longer prefix that narrows the scanned index range down to two rows. With this expression, the database just reads one extra row that is not relevant for the result. The last expression does not have a filter predicate at all: the database just reads the entry that matches the entire `LIKE` expression.



Important

Only the part before the first wildcard serves as an access predicate.

The remaining characters do not narrow the scanned index range—non-matching entries are just left out of the result.

The opposite case is also possible: a `LIKE` expression that starts with a wildcard. Such a `LIKE` expression cannot serve as an access predicate. The database has to scan the entire table if there are no other conditions that provide access predicates.



TIP

Avoid LIKE expressions with leading wildcards (e.g., '%TERM').

The position of the wildcard characters affects index usage—at least in theory. In reality the optimizer creates a generic execution plan when the search term is supplied via bind parameters. In that case, the optimizer has to guess whether or not the majority of executions will have a leading wildcard.

Most databases just assume that there is no leading wildcard when optimizing a LIKE condition with bind parameter, but this assumption is wrong if the LIKE expression is used for a full-text search. There is, unfortunately, no direct way to tag a LIKE condition as full-text search. The box "Labeling Full-Text LIKE Expressions" shows an attempt that does not work. Specifying the search term without bind parameter is the most obvious solution, but that increases the optimization overhead and opens an SQL injection vulnerability. An effective but still secure and portable solution is to intentionally obfuscate the LIKE condition. "Combining Columns" on page 70 explains this in detail.

LABELING FULL-TEXT LIKE EXPRESSIONS

When using the LIKE operator for a full-text search, we could separate the wildcards from the search term:

```
WHERE text_column LIKE '%' || ? || '%'
```

The wildcards are directly written into the SQL statement, but we use a bind parameter for the search term. The final LIKE expression is built by the database itself using the string concatenation operator || (Oracle, PostgreSQL). Although using a bind parameter, the final LIKE expression will always start with a wildcard. Unfortunately databases do not recognize that.

For the PostgreSQL database, the problem is different because PostgreSQL assumes there *is* a leading wildcard when using bind parameters for a LIKE expression. PostgreSQL just does not use an index in that case. The only way to get an index access for a LIKE expression is to make the actual

search term visible to the optimizer. If you do not use a bind parameter but put the search term directly into the SQL statement, you must take other precautions against SQL injection attacks!

Even if the database optimizes the execution plan for a leading wildcard, it can still deliver insufficient performance. You can use another part of the **where** clause to access the data efficiently in that case—see also "Index Filter Predicates Used Intentionally" on page 112. If there is no other access path, you might use one of the following proprietary full-text index solutions.

MySQL
: MySQL offers the **match** and **against** keywords for full-text searching. Starting with MySQL 5.6, you can create full-text indexes for InnoDB tables as well—previously, this was only possible with MyISAM tables. See "Full-Text Search Functions" in the MySQL documentation.

Oracle Database
: The Oracle database offers the **contains** keyword. See the "Oracle Text Application Developer's Guide."

PostgreSQL
: PostgreSQL offers the **@@** operator to implement full-text searches. See "Full Text Search" in the PostgreSQL documentation.

  Another option is to use the WildSpeed[2] extension to optimize `LIKE` expressions directly. The extension stores the text in all possible rotations so that each character is at the beginning once. That means that the indexed text is not only stored once but instead as many times as there are characters in the string—thus it needs a lot of space.

SQL Server
: SQL Server offers the **contains** keyword. See "Full-Text Search" in the SQL Server documentation.



Think about it

How can you index a `LIKE` search that has only one wildcard at the beginning of the search term (`'%TERM'`)?

[2] http://www.sai.msu.su/~megera/wiki/wildspeed

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

INDEX MERGE

It is one of the most common question about indexing: is it better to create one index for each column or a single index for all columns of a **where** clause? The answer is very simple in most cases: one index with multiple columns is better.

Nevertheless there are queries where a single index cannot do a perfect job, no matter how you define the index; e.g., queries with two or more independent range conditions as in the following example:

```
SELECT first_name, last_name, date_of_birth
  FROM employees
 WHERE UPPER(last_name) < ?
   AND date_of_birth    < ?
```

It is impossible to define a B-tree index that would support this query without filter predicates. For an explanation, you just need to remember that an index is a linked list.

If you define the index as `UPPER(LAST_NAME)`, `DATE_OF_BIRTH` (in that order), the list begins with A and ends with Z. The date of birth is considered only when there are two employees with the same name. If you define the index the other way around, it will start with the eldest employees and end with the youngest. In that case, the names only have a minor impact on the sort order.

No matter how you twist and turn the index definition, the entries are always arranged along a chain. At one end, you have the small entries and at the other end the big ones. An index can therefore only support one range condition as an access predicate. Supporting two independent range conditions requires a second axis, for example like a chessboard. The query above would then match all entries from one corner of the chessboard, but an index is not like a chessboard—it is like a chain. There is no corner.

You can of course accept the filter predicate and use a multi-column index nevertheless. That is the best solution in many cases anyway. The index definition should then mention the more selective column first so it can be used with an access predicate. That might be the origin of the "most selective first" myth but this rule only holds true if you cannot avoid a filter predicate.

The other option is to use two separate indexes, one for each column. Then the database must scan both indexes first and then combine the results. The duplicate index lookup alone already involves more effort because the database has to traverse two index trees. Additionally, the database needs a lot of memory and CPU time to combine the intermediate results.



Note

One index scan is faster than two.

Databases use two methods to combine indexes. Firstly there is the index join. Chapter 4, "The Join Operation" explains the related algorithms in detail. The second approach makes use of functionality from the data warehouse world.

The data warehouse is the mother of all ad-hoc queries. It just needs a few clicks to combine arbitrary conditions into the query of your choice. It is impossible to predict the column combinations that might appear in the **`where`** clause and that makes indexing, as explained so far, almost impossible.

Data warehouses use a special purpose index type to solve that problem: the so-called *bitmap index*. The advantage of bitmap indexes is that they can be combined rather easily. That means you get decent performance when indexing each column individually. Conversely if you know the query in advance, so that you can create a tailored multi-column B-tree index, it will still be faster than combining multiple bitmap indexes.

By far the greatest weakness of bitmap indexes is the ridiculous **`insert`**, **`update`** and **`delete`** scalability. Concurrent write operations are virtually impossible. That is no problem in a data warehouse because the load processes are scheduled one after another. In online applications, bitmap indexes are mostly useless.



Important

Bitmap indexes are almost unusable for online transaction processing (OLTP).

Many database products offer a hybrid solution between B-tree and bitmap indexes. In the absence of a better access path, they convert the results of several B-tree scans into in-memory bitmap structures. Those can be combined efficiently. The bitmap structures are not stored persistently but discarded after statement execution, thus bypassing the problem of the poor write scalability. The downside is that it needs a lot of memory and CPU time. This method is, after all, an optimizer's act of desperation.

Partial Indexes

So far we have only discussed which *columns* to add to an index. With *partial* (PostgreSQL) or *filtered* (SQL Server) indexes you can also specify the *rows* that are indexed.



Caution

The Oracle database has a unique approach to partial indexing. The next section explains it while building upon this section.

A partial index is useful for commonly used **`where`** conditions that use constant values—like the status code in the following example:

```
SELECT message
  FROM messages
 WHERE processed = 'N'
   AND receiver  = ?
```

Queries like this are very common in queuing systems. The query fetches all unprocessed messages for a specific recipient. Messages that were already processed are rarely needed. If they are needed, they are usually accessed by a more specific criteria like the primary key.

We can optimize this query with a two-column index. Considering this query only, the column order does not matter because there is no range condition.

```
CREATE INDEX messages_todo
          ON messages (receiver, processed)
```

The index fulfills its purpose, but it includes many rows that are never searched, namely all the messages that were already processed. Due to the logarithmic scalability the index nevertheless makes the query very fast even though it wastes a lot of disk space.

With partial indexing you can limit the index to include only the unprocessed messages. The syntax for this is surprisingly simple: a **where** clause.

```
CREATE INDEX messages_todo
          ON messages (receiver)
       WHERE processed = 'N'
```

The index only contains the rows that satisfy the **where** clause. In this particular case, we can even remove the PROCESSED column because it is always 'N' anyway. That means the index reduces its size in two dimensions: vertically, because it contains fewer rows; horizontally, due to the removed column.

The index is therefore very small. For a queue, it can even mean that the index size remains unchanged although the table grows without bounds. The index does not contain all messages, just the unprocessed ones.

The **where** clause of a partial index can become arbitrarily complex. The only fundamental limitation is about functions: you can only use deterministic functions as is the case everywhere in an index definition. SQL Server has, however, more restrictive rules and neither allow functions nor the OR operator in index predicates.

A database can use a partial index whenever the **where** clause appears in a query.



THINK ABOUT IT

What peculiarity has the smallest possible index for the following query:

```
SELECT message
  FROM messages
 WHERE processed = 'N';
```


EX LIBRIS GHEORGHE GABRIEL SICHIM <GABRIEL.SICHIM@GMAIL.COM>